

Frontend Engineer 경력기술서

더존비즈온 Enterprise SaaS / React Native 모바일 앱

운영 중인 Enterprise SaaS와 React Native 모바일 앱에서 기능 개선, 정책 변경 대응, UI 품질 개선을 중심으로 개발한 경험

1. 전체 개요

도메인	Enterprise SaaS / React Native 모바일 앱
포지션	웹앱 프론트엔드 중심 Frontend Engineer
업무 성격	신규 개발보다 운영 중인 기능 개선과 구조 정리 비중이 높았음
주요 기능	문서 공유, 오픈 링크, 권한 관리, 조직도(Tree), 사용자 검색, 댓글, 멘션, 다중 선택, WebSocket 메신저, 화상회의, AI Chat, OCR, STT, Prompt Library, WebView Bridge, Camera, Gallery, Permission, Deep Link
중점 경험	정책 변경 대응, 조직도 기반 검색, 긴 응답 시간을 가진 기능의 상태 설계, 모바일 UI 분기 대응

운영 서비스에서는 새 기능을 만드는 것보다 이미 있는 기능을 수정하는 일이 훨씬 많았습니다.

기획 변경, QA 수정, 권한 정책 변경이 반복됐기 때문에 화면 하나를 빨리 고치는 것보다 같은 문제가 다시 생기지 않도록 수정 범위를 줄이는 쪽으로 개발했습니다.

실제로도 기능을 추가하는 날보다 기존 화면을 다시 열어보는 날이 더 많았습니다.

2. 권한 관리

1) 프로젝트 개요

운영 중인 문서 공유 서비스에서 권한은 한 번 확인하고 끝나는 값이 아니었습니다.

문서 주소가 바뀌거나 정책이 바뀌면 다시 확인해야 했고, 메뉴 노출, 상세 진입, 버튼 실행도 각각 다르게 봐야 했습니다.

2) 내가 담당한 기능

- 문서 공유 / 오픈 링크 화면 권한 제어
- 역할별 메뉴 노출 여부 정리
- 상세 진입 가능 여부와 버튼 실행 가능 여부 분리
- 주소 변경 및 정책 변경 시 재조회 흐름 정리

3) 실제 문제

처음에는 화면마다 권한 조건을 직접 넣었습니다.

그때는 빨랐습니다. 그런데 정책이 바뀔 때마다 '여기도 수정했나?'를 계속 확인해야 했습니다.

같은 계정인데 목록에서는 버튼이 숨겨지고 상세에서는 버튼이 보이는 식으로 화면마다 동작이 달라지는 일도 생겼습니다.

4) 왜 어려웠는지

권한이 단순히 yes / no 로 끝나지 않았습니다.

메뉴는 보여주되 실행은 막아야 하는 경우도 있었고, 문서 주소가 바뀌면 이전 결과를 그대로 쓰면 안 되는 경우도 있었습니다.

조건문 하나 추가하면 끝날 줄 알았는데, 운영에서는 그 조건문이 계속 남았습니다.

5) 어떻게 해결했는지

처음에는 화면마다 조건문을 손보는 방식으로 대응했습니다.

그런데 정책이 한 번 바뀌고 끝나는 게 아니어서, 결국 권한 판단을 한 곳으로 모으는 편이 다음 변경에도 낫다고 봤습니다.

그래서 화면은 결과만 받고, 판단은 공통 권한 로직에서 하도록 정리했습니다.

- 문서 주소, 정책 키, 사용자 역할이 바뀌면 다시 조회해야 하는 조건을 먼저 정리했습니다.
- 화면에서는 role 비교를 직접 하지 않고 canView, canEdit 같은 결과만 쓰게 바꿨습니다.
- 노출, 진입, 실행, 조회 범위를 한 덩어리로 보지 않고 나눠서 판단했습니다.
- 정책 변경이 오면 먼저 공통 권한 로직과 매핑을 보고, 이후에 화면을 확인하는 순서로 작업 방식을 바꿨습니다.

흐름

사용자
↓
문서 진입
↓
권한 조회
↓
메뉴 노출 여부
↓
버튼 활성화 여부
↓
화면 표시

6) 운영 중 실제 사례

운영 중 권한 정책이 바뀌면서 같은 역할이라도 메뉴는 보여주고 수정 버튼은 숨겨야 하는 요구가 들어왔습니다. 기존 구조에서는 목록, 상세, 모달을 각각 따로 수정해야 했습니다.

QA 에서 같은 계정인데 목록에서는 버튼이 숨겨지고 상세에서는 버튼이 보이는 버그를 잡았습니다. 그때부터는 화면을 먼저 고치기보다 권한 판단 기준부터 확인하는 방식으로 바꿨습니다.

문서 주소가 바뀐 뒤에도 이전 권한 결과를 그대로 쓰는 화면이 있어, 주소 변경 시 재조회 조건을 따로 정리했습니다.

7) 이후 바뀐 방식

이 경험 이후에는 권한 변경 요청이 들어오면 화면부터 수정하지 않고 권한 기준부터 먼저 확인하는 습관이 생겼습니다.

새 화면을 만들 때도 조건문을 바로 넣기보다, 기존 권한 판단 구조에 붙일 수 있는지부터 먼저 보게 됐습니다.

3. 댓글 멘션 / 조직도 기반 사용자 검색

1) 프로젝트 개요

댓글에서 @멘션 기능을 구현했습니다.

멘션 대상은 아마란스 조직도 데이터를 사용했고, 원본 데이터가 Tree 구조라서 검색 결과를 어떤 방식으로 보여 줄지가 핵심이었습니다.

2) 내가 담당한 기능

- 댓글 입력창 @멘션 트리거 처리
- 조직도(Tree) 기반 사용자 검색
- 검색 결과 선택 후 멘션 삽입 흐름 정리
- 다중 선택 및 선택 취소 흐름 반영

3) 실제 문제

처음에는 이름 검색만 붙이면 될 줄 알았습니다.

그런데 실제로는 동명이인도 있었고, 검색 결과가 많아지면 사용자가 원하는 사람을 찾기까지 너무 오래 걸렸습니다.

조직 구조를 완전히 버리면 누군지 헷갈렸고, 반대로 트리를 그대로 보여주면 모바일에서는 너무 무거웠습니다.

4) 왜 어려웠는지

React Native 에서 검색 입력, 리스트, 키보드, 선택 흐름이 한 번에 얹혀 있었습니다.

검색은 평면 리스트가 빠르는데 조직도는 트리라서 정보가 더 많았습니다.

둘 중 하나만 택하면 다른 쪽이 불편해져서 기획과 QA 피드백을 받으며 UI 를 여러 번 바꿨습니다.

5) 어떻게 해결했는지

트리를 그대로 보여주는 방식도 봤지만, 모바일에서는 원하는 사람을 찾는 데 시간이 너무 오래 걸렸습니다.

그래서 결과는 평면 리스트로 보여주되, 부서명과 조직 경로를 같이 붙여 누군지 바로 구분할 수 있게 했습니다.

- 조직도 원본은 유지하되 검색용 데이터 구조를 따로 구성했습니다.
- 검색 결과에는 이름만 보여주지 않고 부서와 조직 경로를 같이 표시했습니다.
- 멘션 선택 후 입력창 커서와 토큰 범위가 어긋나지 않도록 입력 상태를 분리해 관리했습니다.
- 키보드와 리스트가 겹치는 문제 때문에 Bottom Sheet 와 입력창 높이를 여러 번 조정했습니다.

흐름

사용자
↓
댓글 입력
↓
@ 입력 감지
↓
조직도 검색 인덱스 조회
↓
검색 결과 리스트 표시
↓
사용자 선택 후 멘션 삽입

6) 운영 중 실제 사례

QA 에서 동명이인을 구분하기 어렵다는 피드백이 반복적으로 나왔습니다. 이름만 보이던 결과에 부서명과 조직 경로를 붙인 뒤에야 피드백이 줄었습니다.

검색 결과가 많을수록 찾기 어렵다는 기획 의견이 있어 최근 선택 사용자 위치와 결과 정렬 방식을 계속 조정했습니다.

조직도 데이터는 계속 바뀔 수 있어서 검색 결과도 조직도 최신 데이터 기준으로 보여줘야 했습니다. 그래서 검색용 데이터도 원본 조직도 갱신 시점과 맞춰 다시 보게 했습니다.

React Native 에서 키보드가 올라오면 리스트 하단이 가려지는 문제가 있었고, 이 부분은 Safe Area 와 키보드 높이를 같이 보면서 화면을 손봤습니다.

7) 이후 바뀐 방식

이후에는 검색 기능을 볼 때 결과 정확도만 보지 않고, 사용자가 몇 번 내려야 원하는 사람을 찾는지도 같이 보게 됐습니다.

모바일에서는 데이터 구조보다 입력 흐름과 키보드 동작을 먼저 점검하는 습관이 생겼습니다.

4. AI Chat / OCR / STT 상태 흐름 정리

1) 프로젝트 개요

AI Chat, OCR, STT 는 공통적으로 응답이 오래 걸릴 수 있는 기능이었습니다.

프론트엔드에서 서버를 빠르게 만들 수는 없었기 때문에, 사용자가 지금 어떤 상태인지 알 수 있게 만드는 쪽에 집중했습니다.

2) 내가 담당한 기능

- AI Chat 요청/응답 상태 표시
- OCR 업로드 후 처리 상태 표시
- STT 녹음/전송/변환 상태 정리
- 세 기능의 상태 흐름을 비슷한 구조로 맞춤

3) 실제 문제

초기에는 기능마다 로딩 방식이 달랐습니다.

어떤 화면은 Spinner 만 돌고 있었고, 어떤 화면은 실패했는지 기다리는 중인지도 헷갈렸습니다.

사용자는 느리다는 것보다 '지금 뭐가 진행 중인지 모르겠다'는 반응을 더 많이 보였습니다.

4) 왜 어려웠는지

프론트엔드가 응답 속도를 직접 해결할 수 없다는 점이 가장 답답했습니다.

그래서 더더욱 상태를 잘 보여줘야 했습니다.

같은 앱 안에서 AI Chat, OCR, STT 가 서로 다른 방식으로 기다리게 하면 기능마다 신뢰도가 다르게 느껴질 수 있었습니다.

5) 어떻게 해결했는지

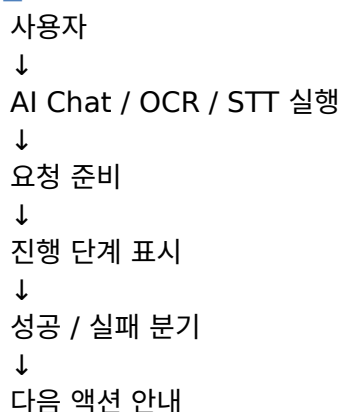
AI 서버 응답 속도는 프론트에서 해결할 수 있는 문제가 아니었습니다.

대신 사용자가 '멈췄다'고 느끼지 않도록 상태와 진행 단계를 보여주는 데 집중했습니다.

그래서 상태 이름은 공통으로 맞추고, 기능별로 보여줄 문구와 액션만 다르게 두는 쪽으로 정리했습니다.

- idle, pending, processing, success, error 처럼 공통 상태를 먼저 정했습니다.
- 세 기능이 같은 상태 이름을 쓰도록 맞췄습니다.
- 업로드 중인지, 분석 중인지, 응답을 기다리는 중인지 단계를 나눠 보여주도록 바꿨습니다.
- 실패 시에는 다시 시도할 수 있는지와 이전 입력을 유지할지까지 같이 정리했습니다.

흐름



6) 운영 중 실제 사례

AI Chat 에서는 응답이 늦을 때 사용자가 전송 버튼을 여러 번 누르는 경우가 있었습니다. 그래서 요청 중이라는 상태를 더 분명하게 보여주고 중복 액션을 막았습니다.

OCR 은 업로드 이후 바로 결과가 나오지 않아 업로드 단계와 분석 단계를 구분해서 보여주도록 바꿨습니다.

STT 는 녹음 종료 후 전송 중인지 변환 중인지 헷갈린다는 피드백이 있었고, 상태 문구를 나눠서 다시 정리했습니다.

7) 이후 바뀐 방식

이후에는 응답이 오래 걸리는 기능을 보면 Spinner 부터 붙이지 않고, 사용자가 지금 어느 단계에 있는지 먼저 나누는 방식으로 보게 되었습니다.

서버가 느린 상황에서도 프론트에서 설명할 수 있는 부분은 끝까지 챙겨야 한다는 기준이 생겼습니다.

5. 모바일 UI / 플랫폼 분기 대응

1) 프로젝트 개요

운영 단계에서 가장 많이 했던 일은 기능 추가보다 UI 수정이었습니다.

같은 화면도 Android 와 iOS 가 다르게 보였고, Safe Area, Keyboard, Bottom Sheet, 해상도 차이 때문에 실제 단말에서 수정이 계속 나왔습니다.

2) 내가 담당한 기능

- Android / iOS 화면 차이 보정
- Safe Area 및 Keyboard 대응
- Bottom Sheet 와 입력창 인터랙션 수정
- Camera / Gallery / Permission / Deep Link 진입 흐름 보정

3) 실제 문제

시뮬레이터에서는 괜찮아 보이는데 실제 단말에서 깨지는 경우가 많았습니다.

키보드가 올라오면 버튼이 가려지고, Bottom Sheet 높이가 달라서 리스트가 잘리고, 권한 팝업 이후 복귀 흐름이 꼬이는 일이 반복되었습니다.

기획에서도 같은 화면의 레이아웃과 인터랙션이 자주 바뀌었습니다.

4) 왜 어려웠는지

모바일 UI 는 코드 한 줄보다 조합이 더 어렵다고 느꼈습니다.

같은 화면도 단말, OS, Safe Area, 키보드, 권한 팝업, 딥링크 진입 여부에 따라 결과가 달랐습니다.

실제로는 신규 기능보다 이런 예외 흐름 수정이 더 자주 들어왔습니다.

5) 어떻게 해결했는지

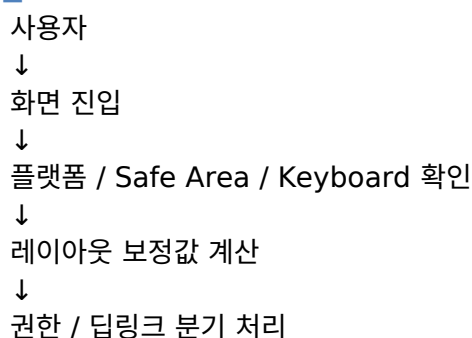
처음에는 화면마다 바로 분기를 넣어 고쳤습니다.

그런데 비슷한 계산이 계속 반복됐고, 같은 이슈가 다른 화면에서도 다시 나왔습니다.

그래서 반복되는 분기만 Hook 과 Utility 로 올리고, 화면 특화 예외는 남기는 쪽으로 정리했습니다.

- Safe Area 와 Keyboard 높이를 합쳐 레이아웃 보정값을 계산하는 부분을 분리했습니다.
- Bottom Sheet, 입력창, 리스트 높이 계산처럼 반복되는 부분은 공통 로직으로 묶었습니다.
- 카메라 / 갤러리 / 권한 팝업 이후 복귀 흐름은 화면 밖으로 옮겨 플랫폼 분기를 줄였습니다.
- 딥링크 진입 시 초기 상태와 권한 확인 순서를 다시 정리했습니다.

흐름





화면 표시

6) 운영 중 실제 사례

QA 에서 특정 Android 기기에서는 키보드가 올라오면 전송 버튼이 가려진다는 이슈가 계속 나왔습니다. 화면마다 임시 margin 을 주는 대신 키보드 높이 기준으로 다시 정리했습니다.

iOS 에서는 Safe Area 때문에 Bottom Sheet 하단 버튼 위치가 어색해지는 경우가 있어 하단 inset 을 따로 반영했습니다.

권한 팝업 이후 카메라나 갤러리로 복귀하면 화면 상태가 초기화되는 문제가 있어, 권한 확인과 실제 액션 시작 시점을 나눠 처리했습니다.

7) 이후 바뀐 방식

이후에는 모바일 UI 이슈가 들어오면 화면부터 바로 고치지 않고, 비슷한 계산이 다른 화면에도 있는지 먼저 확인하는 습관이 생겼습니다.

플랫폼 분기는 화면 안에 쌓아두지 말고, 반복되기 시작하면 밖으로 빼야 한다는 기준이 생겼습니다.

6. 마무리

운영 환경에서는 새로운 기능보다 기존 기능을 안정적으로 유지하고 변경에 빠르게 대응하는 일이 더 많았습니다. 권한 변경, 조직도 검색, 응답 지연, Android / iOS 분기처럼 자주 다시 열어보게 되는 문제들을 겪으면서 화면 하나를 빨리 고치는 것보다 수정 범위와 기준을 먼저 정리하는 방식으로 일하게 됐습니다.

이 경험을 통해 기능 하나를 구현하는 것보다 변경이 들어왔을 때 어디를 수정해야 하는지 먼저 생각하는 개발 습관을 갖게 되었습니다.